
EEE 6778. Applied Machine Learning II



Module 6. Recommendation Systems

Why Recommendation Systems?

Classical Recommendation Methods

Why Deep Learning in RS?

A series of white, stylized lines on the left side of the slide, resembling a circuit board or a series of steps, extending from the top to the bottom.

Why Recommendation Systems?

Why do we need Recommendation Systems?

- They act as personal assistants, filtering out noise in overwhelming digital environments.
- Used in e-commerce (Amazon product recommendations), entertainment (Netflix movie suggestions), and social media (Instagram content feeds).
- **Netflix reports 80% of watched content is from recommendations, showcasing their critical role.**
- Without them, users face decision fatigue and frustration.

A series of white lines on the left side of the slide, consisting of several horizontal lines and a few diagonal lines that create a sense of depth and structure.

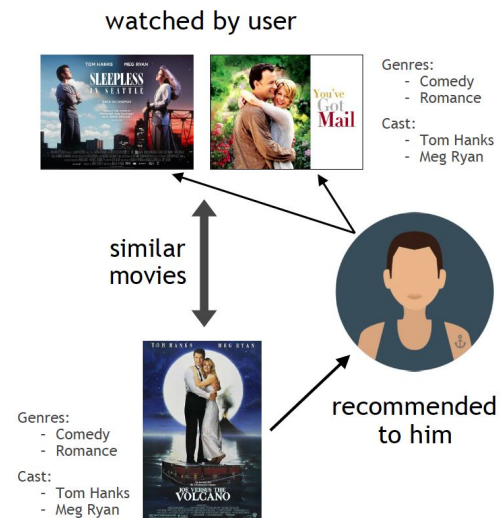
Classical Recommendation Systems

Classical Recommendation Systems

Content Based Filtering

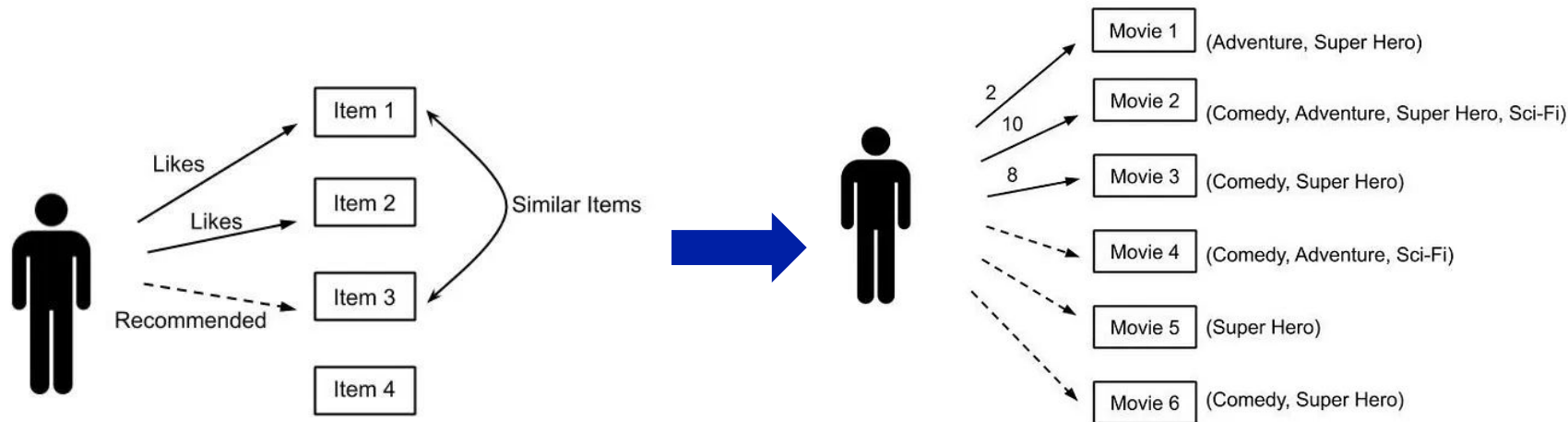
- Recommend items similar to what the user has liked in the past, based on item attributes.
- **Techniques:**
 - TF-IDF / Bag-of-Words for text data (e.g., in movie descriptions).
 - Cosine similarity or Euclidean distance to compare items.
 - Decision trees / Naive Bayes / SVMs to model preferences.
- **Pros:** Personalized. Doesn't require other users' data. Works well when item metadata is rich.
- **Cons:** Hard to capture evolving interests. Limited ability to recommend novel or diverse content (serendipity issue). Cold start problem for new users.

Content-based Filtering



Source: NVIDIA

Foundation (Example)



Consider this following example where the user has given the ratings for 3 movies Movie 1, Movie 2, Movie 3 as 2, 10, 8 respectively. Let's find out what movie from Movie 4, Movie 5, Movie 6 will be recommended to the user.

Source: Levelup

Foundation (Example)

The first step is to make a **one-hot encoded matrix** based on the genres present in a movie. All the genres of the matrix are assigned a column. If a particular genre is present in the Movie it is assigned as 1 otherwise 0.

User's ratings

Movie 1	2
Movie 2	10
Movie 3	8

One Hot encoding Approach

	Comedy	Adventure	Super Hero	Sci-Fi
Movie 1	0	1	1	0
Movie 2	1	1	1	1
Movie 3	1	0	1	0

Foundation (Example)

Now, the user rating matrix is multiplied with the one-hot encoded matrix to form **Weighted Genre Matrix**

	Comedy	Adventure	Super Hero	Sci-Fi
Movie 1	0	2	2	0
Movie 2	10	10	10	10
Movie 3	8	0	8	0

Source: Levelup

Foundation (Example)

This Weighted Genre Matrix is aggregated to form the user profile which is normalized later to help to make a recommendation matrix.

	Comedy	Adventure	Super Hero	Sci-Fi
User	18	12	20	10



	Comedy	Adventure	Super Hero	Sci-Fi
User	0.3	0.2	0.33	0.16

Foundation (Example)

Lastly, the Normalized User Profile is multiplied by the one-hot encoded matrix of the remaining available movies which are not rated by that User and then aggregated to give the recommendation matrix.

	Comedy	Adventure	Super Hero	Sci-Fi
Movie 4	0.3	0.2	0	0.16
Movie 5	0	0	0.33	0
Movie 6	0.3	0	0.33	0



Movie 4	0.66
Movie 5	0.33
Movie 6	0.63

See example in GitHub!!
Resources 
Recommendation Systems

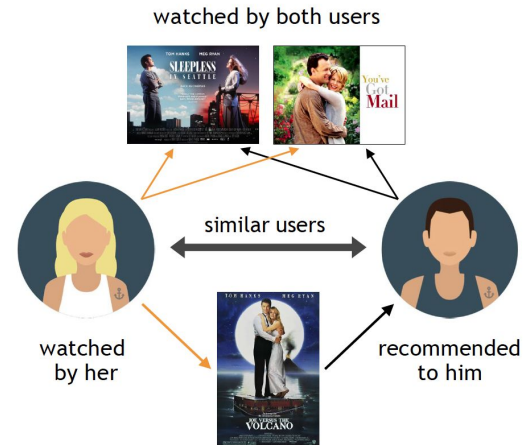
Source: Levelup

Classical Recommendation Systems

Collaborative Filtering (CF)

- Recommend items that similar users liked, or that are similar to items the user liked.
- **Techniques:**
 - Similarity metrics: Cosine similarity, Pearson correlation
 - Neighborhood-based models: k-NN.
- Pros: Doesn't need item metadata. Captures implicit patterns in user behavior.
- Cons: Struggles with sparse data. Cold start for new users/items. Scalability with many users/items.

Collaborative Filtering



Source: NVIDIA

Mathematical Foundation

Matrix Factorization for Recommendation

In this simple user item matrix, Ted and Carol like movies B and C.

Bob likes movie B.

To recommend a movie to Bob, matrix factorization calculates that users who liked B also liked C, so C is a possible recommendation for Bob.

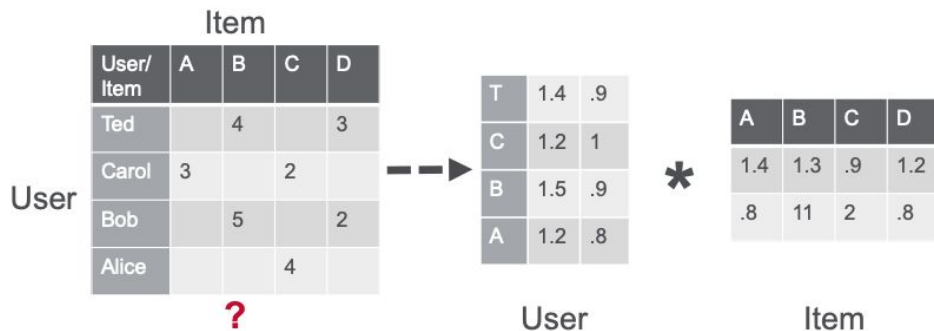
Users		Items		
		A	B	C
	Ted	4	5	5
	Carol		5	5
	Bob		5	?

Source: NVIDIA

Matrix factorization using the **alternating least squares (ALS)** algorithm approximates the sparse user item rating matrix u -by- i as the product of two dense matrices, user and item factor matrices of size $u \times f$ and $f \times i$ (where u is the number of users, i the number of items and f the number of latent features).

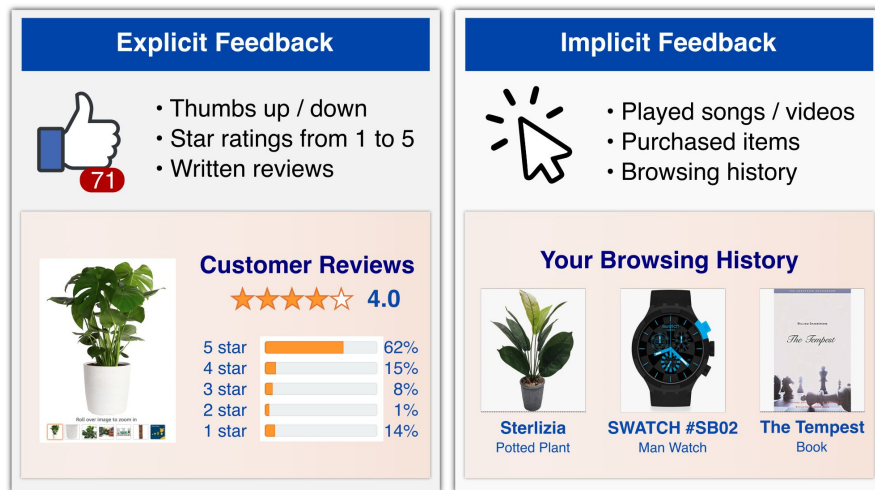
The factor matrices represent latent or hidden features which the algorithm tries to discover. For each user and for each item, the ALS algorithm iteratively learns (f) numeric “factors” that represent the user or item. In each iteration, the algorithm alternatively fixes one factor matrix and optimizes for the other, and this process continues until it converges.

ALS is used to train a latent factor model for collaborative filtering. It works by solving many regularized linear regression problems



Data Collection

- We need to collect user feedback, so that we can have a ground truth for training and evaluating our models. An important distinction has to be made here between explicit feedback and implicit feedback.



Source: Towards
Data Science

Data Collection

- Once we have collected explicit or implicit feedbacks, we can create the user-item rating matrix r_{ui} . For explicit feedback, each entry in r_{ui} is a numerical value
- For implicit feedback, the values in r_{ui} are a boolean values representing presence or lack of interaction
- Notice that the matrix r_{ui} is very sparse, as users interact with few items among all available contents, and they review even fewer items!

How to deal with this sparsity?

Rating Matrix r_{ui}										
Explicit Feedback						Implicit Feedback				
r_{ui} = star rating 1 to 5						r_{ui} = did user watch the movie?				
    						    				
	?	?	4	?	1	1	0	1	0	1
	4	?	?	?	?	1	1	0	1	1
	?	?	?	3	2	0	1	0	1	1
	1	?	?	?	?	1	0	0	1	0

Source: Towards Data Science

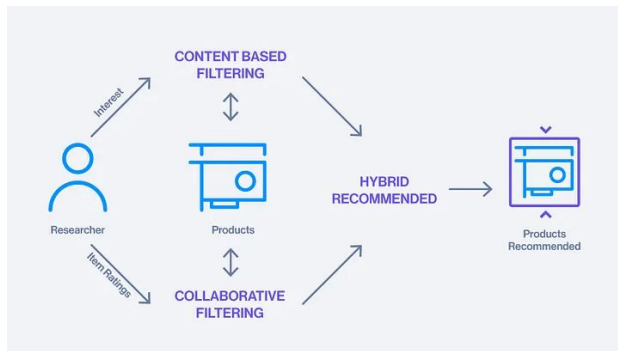


Why Deep Learning in RS?

Classical Recommendation Systems

Hybrid Methods

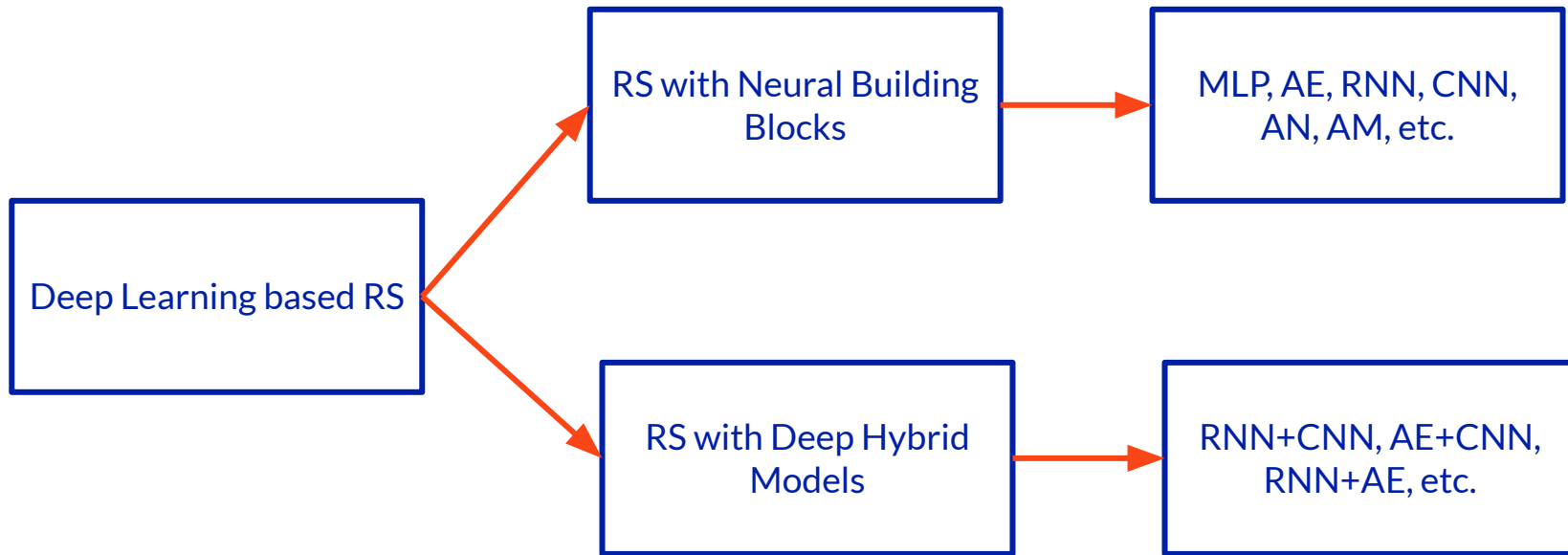
- Recommendations from multiple systems are shown together to the user.
- E.g., Amazon's "Recommended for You" section may include:
 - "Customers who bought this also bought..." (collaborative)
 - "Based on your browsing history..." (content-based)



Why do we need DL in Recommendation Systems?

- DL excels in content-based recommendation involving text, images, or audio
- CNNs capture local patterns in visual product features or review text
- RNNs model temporal dependencies in user sessions or clickstreams
- Multimodal inputs (e.g., tweets + product photos) handled with shared embeddings or fusion networks

Key Deep Learning Models for RS



(Zhang et al., 2019)

Key Deep Learning Models for RS

- **Recommendation with Neural Building Blocks:**
 - MLPs model nonlinear user-item interactions
 - CNNs extract local/global patterns from images, text, and other structured inputs
 - RNNs capture sequential dynamics in sessions, clickstreams, and time-aware content
- **Recommendation with Deep Hybrid Models :**
 - Enables synergy between neural components (e.g., CNN + RNN, AE + MLP)
 - Leverages different strengths (e.g., feature extraction + sequence modeling)
 - Highly flexible; many combinations are underexplored in the literature

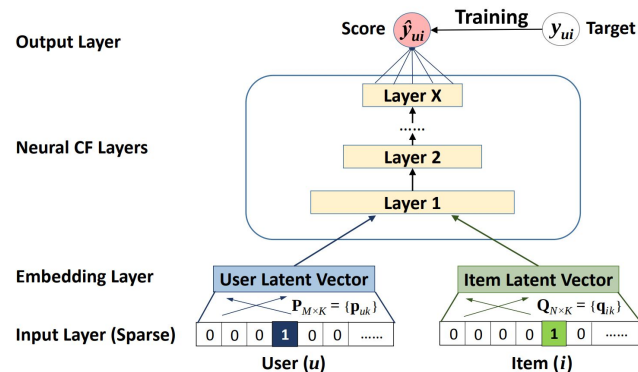
Deep Neural Network-Based Recommendation Models in Specific Application Fields

Thoughts?
Note the multiple possible applications and the number of articles per field

Data Sources/Tasks	Notes	Publications
Sequential Information	w/t User ID	[16, 29, 33, 35, 73, 91, 118, 134, 144, 161, 174, 176, 190, 195, 199, 206]
	Session based w/o User ID	[55–57, 68, 73, 100, 102, 103, 118, 143, 149, 150]
	Check-In, POI	[151, 152, 166, 186]
Text	Hash Tags	[44, 110, 119, 159, 183, 184, 194, 210]
	News	[10, 12, 113, 136, 170, 201]
	Review texts	[11, 87, 127, 147, 175, 198, 203]
	Quotes	[82, 142]
Images	Visual features	[2, 14, 25, 49, 50, 84, 99, 105, 112, 166, 173, 180, 192, 193, 198, 207]
Audio	Music	[95, 154, 168, 169]
Video	Videos	[14, 17, 27, 83]
Networks	Citation Network	[9, 38, 66]
	Social Network	[32, 116, 167]
	Cross Domain	[39, 92, 167]
Others	Cold-start	[155, 157, 171, 172]
	Multitask	[5, 73, 87, 175, 188]
	Explainability	[87, 127]

Neural Collaborative Filtering

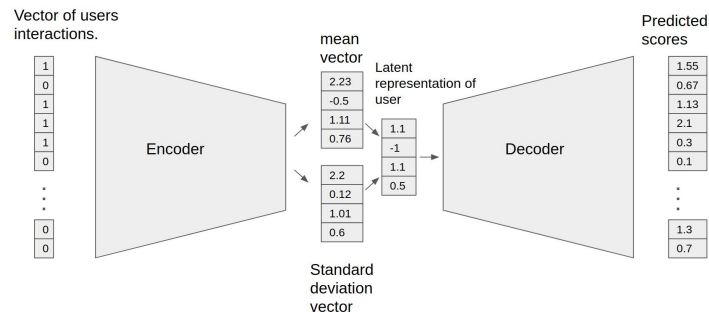
- Neural network that provides collaborative filtering based on user and item interactions.
- The model treats matrix factorization from a non-linearity perspective.
- It takes in a sequence of (user ID, item ID) pairs as inputs, then feeds them separately into a matrix factorization step (where the embeddings are multiplied) and into a multilayer perceptron (MLP) network.
- The outputs of the matrix factorization and the MLP network are then combined and fed into a single dense layer that predicts whether the input user is likely to interact with the input item.



Source: NVIDIA

VAE Applications

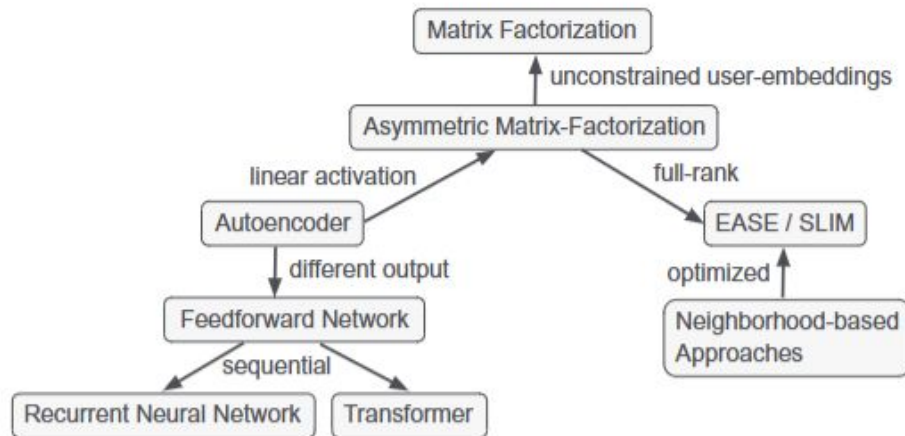
- The model consists of two parts: the encoder and the decoder.
- The **encoder** is a feedforward, fully connected neural network that transforms the input vector, containing the interactions for a specific user, into an n-dimensional variational distribution.
- This variational distribution is used to obtain a latent feature representation of a user (or embedding).
- This latent representation is then fed into the **decoder**, which is also a feedforward network with a similar structure to the encoder. The result is a vector of item interaction probabilities for a particular user.



Hybrid Approaches

Blend architectures for stronger multimodal recommendations.

A Netflix Case Study



(Steck, 2021)



Discussion

Insights, Comments, Questions about the Reading

“There are multiple considerations to balance when developing and deploying machine learning algorithms at a scale of serving hundreds of millions of users. A machine learning model is analogous to a kernel in an operating system in the sense that a lot of work is put into developing the surrounding ecosystem beyond the model itself, so that the model works well. Therefore, it is important to consider practical aspects of the system when considering deep learning.”

Steck et al., 2021, p. 15

Current Challenges

- **Lack of data:** To produce effective predictions, recommender systems require a huge amount of data. Large firms like Google, Apple and Amazon can generate better recommendations since they collect data about their customers on a regular basis.
- **Cold start:** This situation comes when new customers or new items are added to the system. A newly released movie, for example, cannot be recommended to the user until it has received some ratings.
- **Sparsity:** This occurs when the majority of users do not provide ratings or reviews for the items they purchase, causing the rating model to become very sparse, potentially leading to data sparsity issues.

- **Two-Tower / Dual-Encoder Models**
 - **Examples:**
 - YouTube DNN – Deep Neural Network for YouTube Recommendations
 - DSSM – Deep Structured Semantic Model (e.g., Amazon, Microsoft)
 - DLRM – Deep Learning Recommendation Model (Meta/Facebook)
 - **Core Idea:** One tower encodes the **user**, the other the **item**
 - Learn embeddings and use **dot product** or **cosine similarity** for retrieval
- **Multi-Stage Recommendation Pipelines Examples of ranking models:**
 - Wide & Deep - Combines wide linear model and deep neural network
 - DeepFM – Deep Factorization Machine
 - DIN – Deep Interest Network (Alibaba)
 - DCN – Deep & Cross Network
 - **Pipeline Stages:** Candidate retrieval, Ranking (personalized scoring)
Re-ranking (post-processing: diversity, fairness)

- **Transformer-Based Sequential Models**
 - SASRec- Self-Attentive Sequential Recommendation
 - BERT4Rec – Bidirectional Encoder Representations from Transformers for Recommendation
 - Recformer – Transformer-based Recommender System
 - M6-Rec – Multimodal Multitask Mega-transformer for Recommendation
 - TALLRec – Transformer with Alternating Layerwise Learning for Recommendation
 - **Use Cases:** Session-based, sequential behavior, context-aware recsys
- **Graph Neural Network (GNN)-Based Models**
 - PinSage – Pinterest’s Graph Convolutional Recommendation System
 - LightGCN – Lightweight Graph Convolutional Network
 - NGCF – Neural Graph Collaborative Filtering
 - GAT4Rec – Graph Attention Network for Recommendation
 - **Core Concept:** Represent user-item interactions as a graph

- **Multi-Task & Multimodal Learning Models**
 - MMoE – Multi-gate Mixture-of-Experts
 - PLE – Progressive Layered Extraction
 - UniSRec – Unified Sequential Recommendation
 - **Strengths:** Learn multiple objectives (e.g., click-through rate, dwell time and fuse inputs from multiple modalities: text, images, metadata, etc.
- **Contrastive & Self-Supervised Learning for RecSys**
 - CL4Rec – Contrastive Learning for Recommendation
 - S3-Rec – Self-Supervised Sequential Recommendation
 - DuoRec – Dual Contrastive Learning for Recommendation
 - **Techniques:** Data augmentation, positive/negative sampling, view contrast
Improves generalization and robustness